PRISM-S Solution Options Matrix

MediSched AI

PRISM Phase: S – Intentional Solutioning

SmartX Technologies

Date: April 2026

PRISM Phase: S – Structured Design
Purpose: Translate the selected solution into a clear, buildable, reviewable design. Any competent developer should be able to implement this design as written.

1. Executive Summary

1.1 Problem Recap

Diagnostic centers currently rely on manual, phone-based appointment scheduling with no centralized patient registry or structured doctor availability management. This leads to high no-show rates, double bookings, inefficient staff effort, and lack of operational analytics.

1.2 Chosen Solution Summary

To address these challenges, MediSched AI is designed as a centralized web-based platform that enables patient management, doctor slot scheduling, real-time calendar booking, AI-powered outbound calling, call transcription, automated reminders, and analytics.

The selected solution uses:

- Django REST Framework for backend services
- React.js for frontend UI
- PostgreSQL for relational data management
- Celery + Redis for asynchronous processing (calling, transcription, reminders)
- Django Channels for real-time updates

This architecture ensures strong data consistency, modular design, and scalability while satisfying all functional and non-functional requirements.

1.3 Key Trade-offs Accepted

- **Higher setup complexity vs better maintainability:** Django + Celery + Redis introduces more components compared to simpler stacks, but provides a robust and scalable architecture.
- **Separate frontend and backend vs flexibility:** Using React (SPA) and Django (API) requires coordination, but allows independent development and better UI scalability.

- **Synchronous Django ORM vs async performance:** Django's ORM is not fully async, but reliability and strong relational modeling are prioritized over raw performance.
- **Operational overhead (Celery, Redis) vs background processing efficiency:** Additional services increase deployment complexity but are necessary for handling AI calls, retries, and transcription asynchronously.

2. References

Link related PRISM artifacts:

2.1 Problem Brief

The Problem Brief defines the core problem MediSched AI aims to solve.

It highlights that diagnostic centers currently rely on manual and phone-based workflows for appointment scheduling, patient management, and doctor availability coordination. There is no centralized system for maintaining patient records or managing doctor slots, leading to fragmented data and inefficient operations.

Key issues identified include:

- High patient no-show rates (15–30%) due to lack of timely reminders
- Significant staff time (2–4 hours/day) spent on manual outbound calls
- Frequent double-bookings and scheduling conflicts
- No centralized patient registry or call documentation
- Lack of analytics for operational insights

The document also defines business objectives such as reducing manual effort, improving appointment utilization, enabling AI-based calling, and establishing a centralized system for patient and doctor data.

2.2 Derived Perspectives from Requirements

2.2.1 Administrative Perspective

The system design considers administrative needs such as full operational control, visibility into scheduling activities, and access to analytics. It ensures centralized management of doctors, users, and system-wide configurations while enabling monitoring of performance metrics like appointment utilization and call outcomes.

2.2.2 Operational (Staff) Perspective

The design emphasizes efficient day-to-day operations handled by staff, including patient record management, appointment booking, and handling exceptions such as failed AI calls. The system supports quick data entry, search functionality, and real-time scheduling to reduce manual effort and improve productivity.

2.2.3 Doctor Interaction Perspective

The system provides a simplified interface for doctors to manage their availability independently. It focuses on enabling doctors to publish, modify, and block consultation slots while restricting access to unrelated data, ensuring role-based isolation and ease of use.

2.2.4 System & Patient Interaction Perspective

The design incorporates automated system-driven interactions such as AI-based calling, transcription, and reminders to enhance patient engagement. It ensures seamless communication with patients, supports automated booking workflows, and maintains accurate records of all interactions for future reference and analysis.

2.3 Solution Options Matrix

The Solution Options Matrix evaluates multiple technical approaches and selects the most suitable one.

2.3.1 Option 1: Node.js + Next.js + MongoDB

Characteristics:

- Full JavaScript/TypeScript stack
- Flexible schema using MongoDB
- Fast development and unified language across frontend/backend

Limitations:

- Weak relational data handling (not ideal for scheduling system)
- No built-in admin panel
- Higher risk of data inconsistency

2.3.2 Option 2: Django + React + PostgreSQL (Selected)

Characteristics:

- Django REST Framework backend with strong structure
- React frontend for UI
- PostgreSQL for relational integrity
- Celery + Redis for background processing

Advantages:

- Strong data consistency with relational database
- Built-in admin panel for rapid development
- Mature ecosystem for AI and backend processing
- Well-suited for scheduling and transactional systems

This option was selected due to its balance between reliability, scalability, and development speed.

2.3.3 Option 3: FastAPI + Next.js + PostgreSQL

Characteristics:

- High-performance async backend using FastAPI
- Modern architecture with async support

Limitations:

- No built-in admin panel
- Higher development effort for building structure
- Less suitable for rapid prototyping within 8-week timeline

3. Design Goals & Non-Goals

List explicit goals and what this design intentionally does NOT address.

3.1 Design Goals

- **Centralized Data Management:** Provide a single source of truth for patient records, doctor availability, appointments, and call logs to eliminate fragmented data handling.
- **Efficient Appointment Scheduling:** Enable real-time calendar-based booking with conflict detection and prevention of double-booking.
- **Reduction of Manual Effort:** Automate repetitive tasks such as patient calling, reminders, and follow-ups to reduce staff workload.
- **Improved Patient Engagement:** Use AI-based outbound calling, SMS, and email reminders to reduce no-show rates and improve communication.
- **Scalable and Modular Architecture:** Design the system using modular components (API, workers, database, real-time layer) to support future scaling and enhancements.
- **Strong Data Consistency and Integrity:** Use a relational database to ensure accurate relationships between patients, doctors, slots, and appointments.
- **Role-Based Access Control:** Enforce strict access permissions for Admin, Staff, Doctor, and Read-only users to ensure data security.
- **Asynchronous Processing for Background Tasks:** Handle long-running tasks such as calling, transcription, and reminders asynchronously to maintain system responsiveness.
- **Real-Time Updates:** Ensure immediate propagation of scheduling changes across all users through real-time communication mechanisms.
- **Operational Visibility through Analytics:** Provide dashboards and reports for monitoring appointment utilization, call performance, and system efficiency.

3.2 Non-Goals

The system intentionally does NOT address the following:

- **Integration with External EHR/EMR Systems:** The system operates as a standalone platform without integration into existing hospital systems.
- **Patient-Facing Application:** Patients do not directly interact with a web or mobile application; communication is handled via calls, SMS, and email.
- **Billing and Payment Processing:** Financial operations such as billing, invoicing, or payment gateways are out of scope.

- **Advanced Clinical Features:** The system does not include diagnosis, prescription management, or medical decision support.
- **Multi-Branch / Multi-Tenant Support:** The design is limited to a single diagnostic center in the prototype phase.
- **Full Regulatory Compliance Implementation:** While basic security practices are followed, full compliance certifications are not implemented in this phase.
- **Multi-Language AI Support:** The system currently supports a single language for AI-based calling.
- **Advanced Predictive Analytics / Machine Learning Models:** Only basic analytics are included; advanced predictive modeling is not implemented.

4. High-Level Architecture

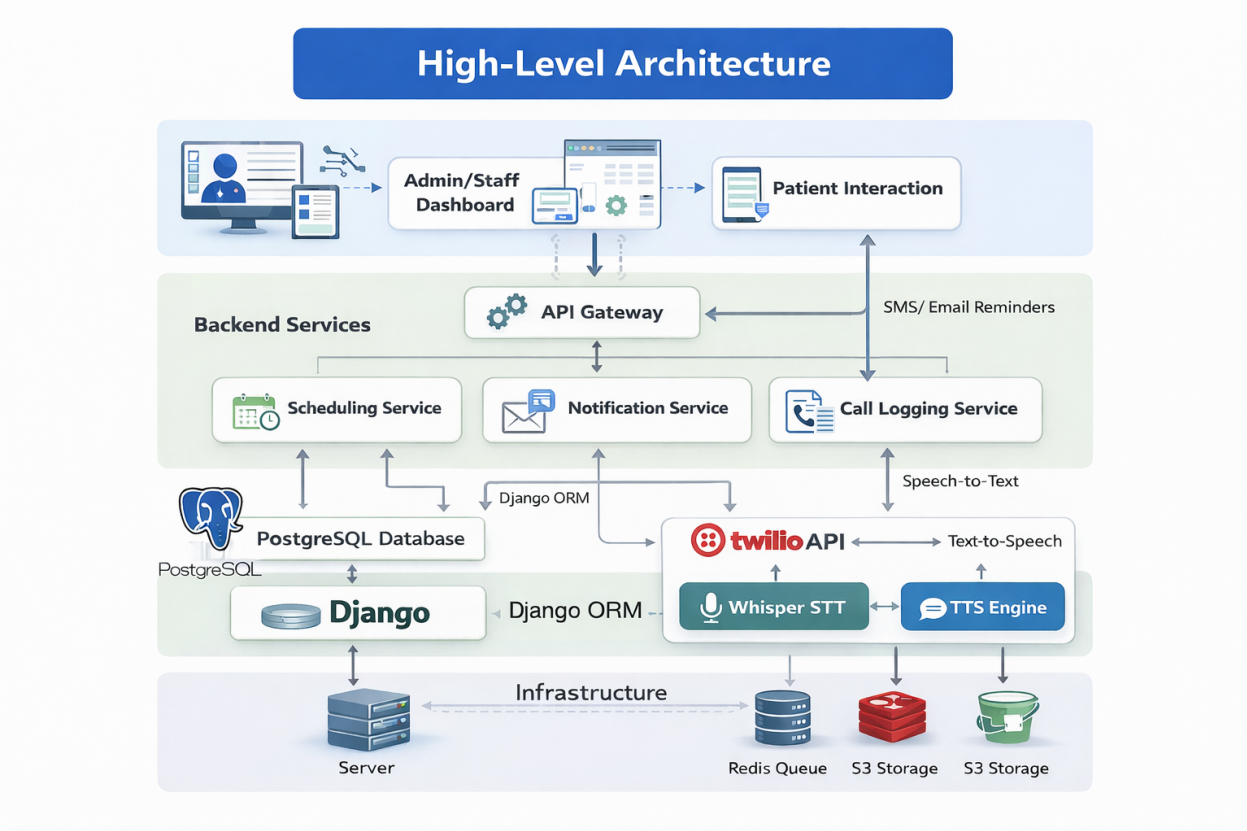


Figure 1: High-Level Architecture

4.1 Components

- User interface layer includes Admin/Staff dashboard and patient interaction channels.
- API Gateway acts as the central entry point for all requests.
- Backend services include Scheduling, Notification, and Call Logging services.
- PostgreSQL database is used for structured data storage.
- Django ORM is used for interaction between backend and database.
- External services include Twilio, Whisper (Speech-to-Text), and Text-to-Speech (TTS).
- Infrastructure components include servers, Redis queue, and S3 storage.

4.2 Responsibilities

- Frontend enables user interaction and triggers system actions.
- API Gateway handles request routing, validation, and authentication.
- Backend services implement core business logic such as scheduling, notifications, and call tracking.
- Database ensures consistent storage and retrieval of system data.
- AI services handle automated calling, speech processing, and responses.
- Infrastructure ensures scalability, performance, and reliable execution.

4.3 Interaction Overview

- Users interact with the system through the dashboard.
- Requests are sent to the API Gateway for processing.
- Backend services execute business logic and process requests.
- Data is stored and retrieved from the database.
- External services are used for calls, messaging, and speech processing.
- Background tasks are handled asynchronously using Redis.
- Results and updates are delivered back to users in real-time.

5. Component-Level Design

Detailed description of each component including responsibilities, inputs/outputs, and dependencies.

5.1 Responsibilities

Component	Responsibilities
Frontend (Dashboard)	Provides user interface for managing patients, appointments, and system operations.
API Gateway	Handles request routing, validation, and authentication.
Scheduling Service	Manages doctor slots, booking, and conflict detection.
Notification Service	Sends reminders and alerts via SMS and email.
Call Logging Service	Tracks call activities and stores call outcomes.
Database (PostgreSQL)	Stores structured data and maintains relationships.
Django Backend (ORM)	Implements business logic and manages database interactions.
Twilio API	Handles voice calls and messaging communication.
Whisper (STT)	Converts speech input into text.
TTS Engine	Converts system text into voice responses.
Redis Queue	Handles asynchronous task processing.
S3 Storage	Stores recordings and related files.
Server / Infrastructure	Hosts application and ensures system availability.

Table 1: Component Responsibilities

5.2 Inputs / Outputs

Component	Inputs	Outputs
Frontend	User actions	API requests, UI display
API Gateway	HTTP requests	Routed service responses
Scheduling Service	Booking requests	Appointment confirmations
Notification Service	Event triggers	SMS/Email notifications
Call Logging Service	Call data	Logs and analytics data
Database	Data queries	Stored/retrieved data
Django Backend	API requests	Processed responses
Twilio API	Call/SMS requests	Communication results
Whisper (STT)	Audio input	Transcribed text
TTS Engine	Text input	Voice output
Redis Queue	Background tasks	Processed jobs
S3 Storage	File uploads	File retrieval
Server	Requests	System responses

Table 2: Component Inputs and Outputs

5.3 Dependencies

Component	Dependencies
Frontend (Dashboard)	API Gateway
API Gateway	Backend Services
Scheduling Service	Database, API Gateway
Notification Service	Twilio API, Backend
Call Logging Service	Twilio API, Whisper, Database
Database (PostgreSQL)	Django ORM
Django Backend (ORM)	PostgreSQL
Twilio API	Backend Services
Whisper (STT)	Call data (Twilio)
TTS Engine	Backend, Twilio
Redis Queue	Backend Services
S3 Storage	Backend Services
Server / Infrastructure	All components

Table 3: Component Dependencies

6. Data Model & Persistence

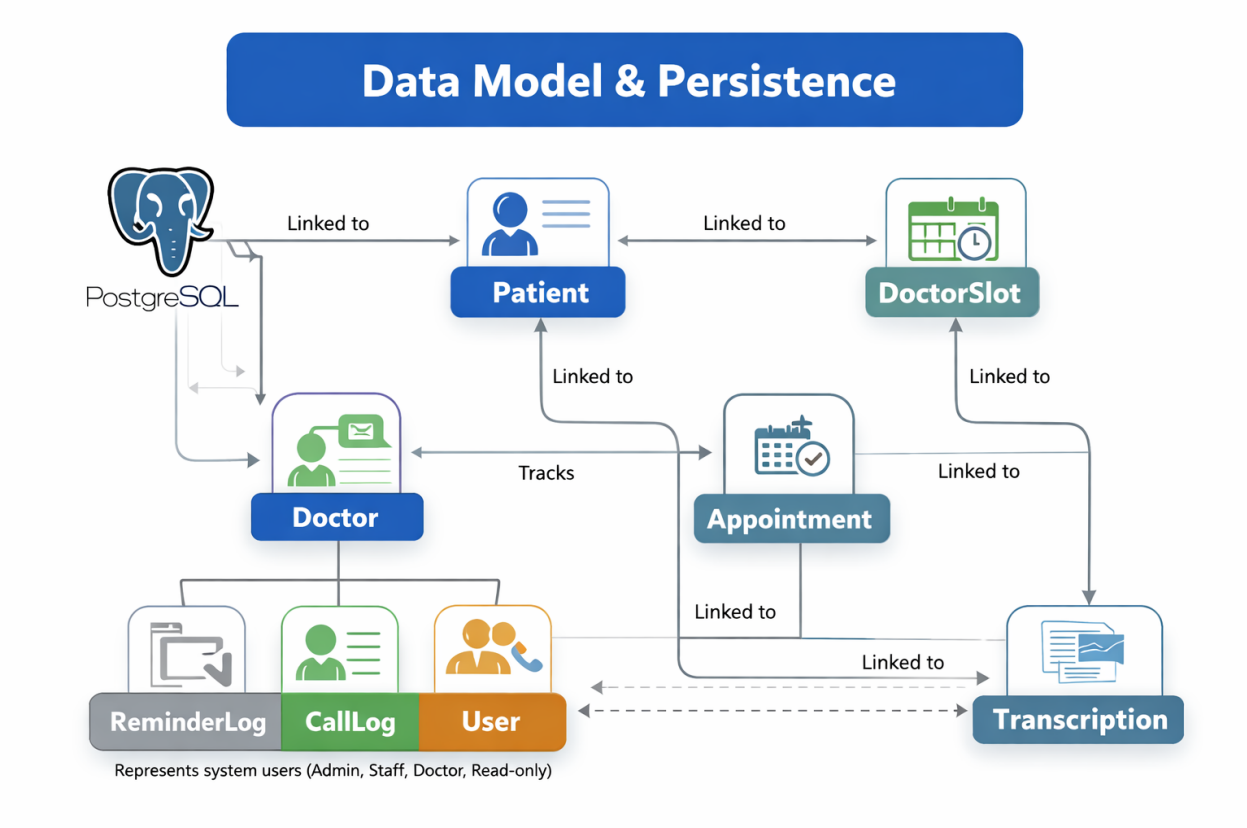


Figure 2: Data Model and Persistence Diagram

This section describes the core data structures, schemas, and storage mechanisms used in the system.

6.1 New Entities

The system introduces several core entities to support scheduling, calling, and analytics functionalities:

- **Patient:** Stores patient details such as name, phone number, date of birth, gender, and optional information like email, address, and medical notes. Acts as the central entity for all interactions.
- **Doctor:** Maintains doctor profile information including name, specialization, contact details, and active/inactive status.

- **DoctorSlot:** Represents individual time slots created by doctors for consultation. Each slot contains date, time, duration, and status (Available, Booked, Blocked).
- **Appointment:** Links patients to doctor slots. Stores booking status, timestamps, and references to both patient and slot.
- **CallLog:** Records details of all outbound calls including patient reference, attempt number, call duration, outcome, and timestamps.
- **Transcription:** Stores speech-to-text output of calls, linked to CallLog and Appointment for traceability and editing.
- **ReminderLog:** Tracks SMS and email reminders sent to patients along with delivery status (sent, delivered, failed).
- **User:** Represents system users (Admin, Staff, Doctor, Read-only) with authentication credentials and role definitions.

6.2 Changes to Existing Entities

Since the system is designed as a greenfield application, there are no legacy entities to modify.

However, internal relationships are structured as follows:

- Patient is linked to multiple Appointments
- Doctor is linked to multiple DoctorSlots
- DoctorSlot is linked to a single Appointment (when booked)
- Appointment is linked to CallLogs and Transcriptions
- User entity enforces role-based access across all modules

These relationships ensure data consistency and enable traceability across workflows.

6.3 Migration Considerations

The system uses PostgreSQL with Django ORM, enabling structured schema management through migrations.

Key considerations include:

- **Schema Evolution:** Changes to models are handled using Django migration tools, ensuring version-controlled database updates.

- **Data Integrity:** Foreign key constraints and unique fields (e.g., patient phone number) are enforced at the database level.
- **Initial Data Setup:** Seed data may be required for roles, admin users, and basic configurations during initial deployment.
- **Scalability:** Database indexing (on fields like phone number, slot time, and doctor ID) ensures efficient querying as data grows.
- **Backup and Recovery:** Regular database backups are required to prevent data loss and support recovery in case of failures.

7. API Design & Contracts

7.1 Endpoints

Endpoint	Method	Description
/api/patients	POST	Creates a new patient record by validating input data and storing it in the database.
/api/patients	GET	Retrieves a list of patients with filtering and search support.
/api/doctors	GET	Fetches doctor details including specialization and availability status.
/api/slots	POST	Creates doctor slots based on date, time range, and duration.
/api/appointments	POST	Books an appointment by linking patient and slot with conflict validation.
/api/appointments	GET	Retrieves scheduled appointments for tracking and management.
/api/calls/initiate	POST	Initiates AI-based outbound calling using integrated communication services.
/api/transcriptions	GET	Retrieves transcribed call data for review and editing.
/api/reminders/send	POST	Triggers automated reminders via SMS or email.

Table 4: API Endpoints

7.2 Request / Response Models

API	Request Model	Response Model
Create Patient	JSON with name, phone, DOB, gender	Success message with patient ID and stored data
Create Slot	JSON with doctor ID, date, time range, duration	Generated slots with status (Available/Blocked)
Book Appointment	JSON with patient ID and slot ID	Confirmation with appointment details and status
Initiate Call	JSON with patient contact and appointment details	Call status (initiated, failed, completed)
Send Reminder	JSON with patient ID and message type	Delivery status (sent, delivered, failed)

Table 5: Request and Response Models

7.3 Error Handling

Error Type	HTTP Code	Handling Strategy
Validation Error	400	Invalid input data; handled by validating fields and returning clear error messages.
Unauthorized Access	401	Authentication failure; resolved using secure token-based authentication.
Conflict Error	409	Double booking issue; prevented using conflict detection logic.
Not Found Error	404	Resource not found; handled with appropriate error response.
Server Error	500	Unexpected failure; logged and handled with fallback response.

Table 6: API Error Handling

7.4 Idempotency and Versioning

Aspect	Description
Idempotency	Ensures repeated API requests produce the same result without duplication. For example, booking APIs use unique request identifiers to prevent multiple bookings for the same slot.
Safe Methods	GET requests are inherently idempotent and used for data retrieval without side effects.
Versioning Strategy	APIs are versioned using URL-based versioning (e.g., /api/v1/), allowing backward compatibility and smooth upgrades.
Backward Compatibility	New versions maintain support for older clients while introducing enhancements without breaking existing functionality.

Table 7: Idempotency and Versioning

8. Workflow & Sequence Diagrams

This section describes the key workflows and sequence of operations in the system.

8.1 Appointment Creation Flow

- Admin/Staff logs into the dashboard and enters patient details.
- The frontend sends a request to the API Gateway (`/api/appointments`).
- The API Gateway validates the request and routes it to the Scheduling Service.
- The Scheduling Service checks slot availability and prevents conflicts.
- Appointment details are stored in the database via Django ORM.
- A confirmation response is sent back to the dashboard.

8.2 Reminder Notification Flow

- After appointment creation, a reminder trigger is generated.
- The backend calls the Notification Service (`/api/reminders/send`).

- The Notification Service processes the request and prepares message content.
- The system integrates with external services (SMS/Email or Twilio).
- Reminder is delivered to the patient.
- Delivery status is stored in the database for tracking.

8.3 AI Call Initiation Flow

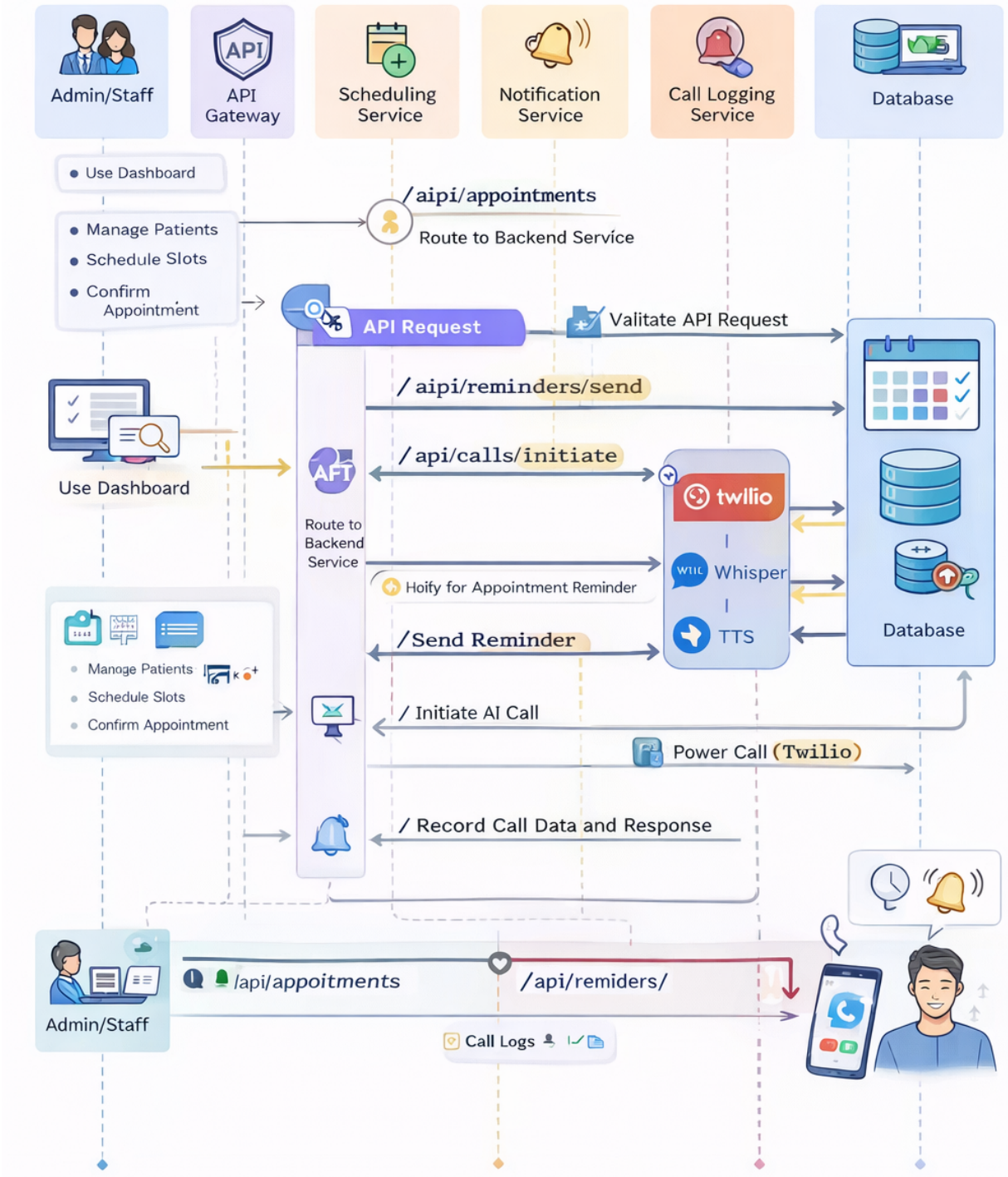
- The system initiates a call using `/api/calls/initiate`.
- The request is routed through the API Gateway to backend services.
- The Twilio API is triggered to place the call to the patient.
- The TTS Engine generates voice prompts.
- The patient responds during the call.

8.4 Speech Processing & Logging Flow

- Patient voice input is captured during the call.
- Audio is sent to Whisper (Speech-to-Text) for transcription.
- Transcribed text is processed by the system.
- Call outcome and transcription are stored in the database.
- The Call Logging Service records call status and analytics.

8.5 End-to-End System Flow

- User actions from the dashboard trigger API requests.
- Requests pass through the API Gateway to backend services.
- Backend services interact with the database for storage and retrieval.
- External services (Twilio, TTS, Whisper) handle communication and AI processing.
- Background tasks such as calls and reminders are handled asynchronously.
- Final results are reflected back to users for monitoring and tracking.


Admin/Staff
API Gateway
Scheduling Service
Notification Service
Call Logging Service
Database
Use Dashboard
Manage Patients
Schedule Slots
Confirm Appointment
/aipi/appointments
Route to Backend Service
API Request
Valitate API Request
/aipi/reminders/send
/api/calls/initiate
Use Dashboard
Route to Backend Service
twllio
Whisper
TTS
Hoify for Appointment Reminder
Database
/Send Reminder
Manage Patients
Schedule Slots
Confirm Appointment
/ Initiate AI Call
Power Call (Twilio)
/ Record Call Data and Response
Admin/Staff
/api/appoitments
/api/remiders/
Call Logs

9. Non-Functional Considerations

Aspect	Description
Performance	The system handles concurrent requests efficiently with optimized database queries and indexing. Asynchronous processing using Redis and Celery ensures non-blocking execution of calls and reminders.
Security	Implements role-based access control (RBAC) and secure authentication. Sensitive data is protected using encryption and secure communication protocols (HTTPS).
Availability	Designed for high availability with reliable infrastructure and minimal downtime. Backup mechanisms ensure data recovery in case of failures.
Scalability	Supports horizontal scaling of services and database. Redis enables scalable background processing, and S3 storage handles large data efficiently.
Observability	Logging and monitoring mechanisms track API activity, errors, and system performance. Call logs and analytics provide operational insights.
Compliance	Ensures secure handling of patient data with restricted access and auditing. Data storage and transmission follow privacy and protection standards.

Table 8: Non-Functional Requirements (NFRs)

10. Failure Modes & Edge Cases

Failure / Edge Case	Handling Strategy
Double booking of slots	Prevented using conflict detection and transactional locking; duplicate requests are rejected with proper error response.
Invalid or missing input data	API-level validation ensures required fields are present; descriptive error messages are returned to guide correction.
Duplicate patient entries	Unique constraints (e.g., phone number) prevent duplication; existing records are reused instead of creating new ones.
AI call not answered	System retries after defined intervals; if unsuccessful, escalates to manual follow-up by staff.
Call failure (network/service issue)	Call attempts are logged and retried using retry policies; fallback to SMS/email if needed.
Incorrect speech transcription	Provides editable transcription interface; maintains logs for traceability and correction.
Reminder delivery failure	Retries notification delivery; logs failed attempts and allows manual resend.
Database connection failure	Returns safe error response; retries connection and logs failure for monitoring.
Unauthorized access attempt	Blocked using authentication and RBAC; suspicious activities are logged for auditing.
Session timeout / token expiry	Requires re-authentication; ensures secure session handling without exposing data.
Concurrent updates on same record	Managed using database transactions and locking to maintain consistency.
External API failure (Twilio, STT)	Retry mechanism with fallback options; system continues core operations without full dependency failure.
High system load	Load balancing and queue-based processing (Redis) ensure system stability under heavy usage.
Data inconsistency or corruption	Enforced using constraints and validation checks; periodic backups ensure recovery.
Server downtime or crash	Backup and recovery mechanisms restore system; minimal downtime ensured through reliable infrastructure.

Table 9: Failure Modes and Edge Case Handling

11. Test Strategy

11.1 Unit Tests

Component	Test Case	Validation	Example
Patient Module	Create patient with valid data	Ensures patient record is saved correctly	Input: name, phone → Output: patient ID generated
Patient Module	Duplicate phone number	Prevents duplicate patient entry	Same phone entered twice → Error shown
Slot Module	Generate slots	Verifies correct slot creation	Input: 10AM–12PM → Output: multiple slots
Slot Module	Invalid time range	Rejects incorrect slot input	End time ¡ start time → Error
Appointment Module	Book appointment	Confirms valid booking	Patient + slot → Appointment created
Appointment Module	Double booking check	Prevents duplicate booking	Same slot → Conflict error
Notification Module	Send reminder trigger	Ensures reminder is generated	Appointment created → Reminder queued
Call Module	Call initiation	Verifies call request creation	Valid phone → Call request sent

Table 10: Unit Test Cases

11.2 Integration Tests

Integration Flow	Test Case	Validation	Example
Frontend + API Gateway	Request routing	Ensures frontend requests reach backend correctly	Click booking → API triggered
API + Scheduling Service	Slot validation	Ensures slot availability check works	Slot selected → Valid/invalid response
Backend + Database	Data persistence	Ensures data is stored and retrieved correctly	Appointment saved → Fetch works
Backend + Notification Service	Reminder integration	Ensures reminders are triggered	Booking → SMS sent
Backend + Twilio API	Call integration	Ensures call is initiated properly	Call API → Twilio call triggered
Call + STT (Whisper)	Transcription flow	Ensures speech converts to text	Audio → Text output
Backend + Redis	Async processing	Ensures background tasks are executed	Reminder queued → Executed

Table 11: Integration Test Cases

11.3 End-to-End Tests

Workflow	Test Case	Validation	Example
Patient Registration → Booking	Complete booking flow	Ensures patient creation to booking works end-to-end	Add patient → Book slot → Success
Booking → Reminder	Reminder workflow	Ensures reminders are sent after booking	Booking done → SMS received
Booking → AI Call	Call workflow	Ensures automated call is triggered	Appointment → Call initiated
Call → Transcription	Speech processing flow	Ensures call is transcribed correctly	Call → Text stored
Dashboard → Analytics	Data tracking	Ensures logs reflect system activity	Call made → Data visible
Failure Handling Flow	Retry mechanism	Ensures retry and fallback works	Call failed → Retry triggered
Full System Workflow	End-to-end system validation	Ensures all modules work together	Booking → Reminder → Call → Logs

Table 12: End-to-End Test Cases

12. Deployment, Rollout & Rollback

12.1 Overview

MediSched AI will be deployed using Docker to ensure consistency between development and production environments.

12.2 Deployment Architecture

Services are containerized and managed using Docker Compose:

- Frontend: React application
- Backend: Django REST API
- Database: PostgreSQL
- Worker: Celery for background tasks

- Cache/Queue: Redis for asynchronous processing

Possible deployment platforms include:

- Prototype: Railway, Render, or VPS
- Production: AWS, GCP, or Azure

12.3 Rollout Strategy

- Phase 1: Patient management, scheduling, and calendar features
- Phase 2: AI calling, transcription, and analytics features

12.4 Backward Compatibility

- API versioning (e.g., /api/v1/) is used to maintain compatibility
- Ensures new updates do not break existing functionality

12.5 Rollback Plan

If deployment fails or critical issues occur:

- Revert to the previous stable Docker image
- Restore database from backup
- Disable faulty features using feature flags

12.6 Monitoring & Health Checks

- Health endpoint (e.g., /health) is used to check system status
- Monitor API, worker, and database connectivity
- Log errors, task failures, and system latency for analysis

13. Risks & Mitigations

- **Low patient adoption of AI calls:** Patients may ignore or distrust automated calls. *Mitigation:* Use natural-sounding TTS, provide human fallback, and allow opt-out options.
- **Speech-to-Text inaccuracies:** Errors in transcription due to accents or medical terms. *Mitigation:* Fine-tune models, allow manual correction, and store editable transcripts.

- **Twilio limitations or failures:** Call service limitations or downtime. *Mitigation:* Implement retry mechanisms and fallback to SMS/email notifications.
- **Double booking conflicts:** Multiple users booking the same slot simultaneously. *Mitigation:* Use transaction locking and conflict detection logic.
- **High system load:** Performance degradation under heavy usage. *Mitigation:* Use scalable architecture, load balancing, and Redis-based queue processing.
- **Data security breach:** Unauthorized access to sensitive patient data. *Mitigation:* Implement RBAC, encryption, and secure authentication mechanisms.
- **External service dependency failure:** Failure of APIs like Twilio or STT services. *Mitigation:* Use retry logic, fallback strategies, and graceful degradation.
- **Database failure or data loss:** Risk of losing critical data. *Mitigation:* Maintain regular backups and implement recovery mechanisms.
- **Deployment failure:** Issues during release causing downtime. *Mitigation:* Use rollback strategy, Docker versioning, and staged deployment.
- **Scope creep:** Additional features extending beyond timeline. *Mitigation:* Strict scope control, phased rollout, and requirement prioritization.

14. Open Questions

- What is the maximum number of concurrent users the system should support?
- How should patient consent be handled for AI-based calling?
- What retry strategy should be used for failed calls and notifications?
- How will call scheduling be prioritized when multiple calls are queued?
- What level of accuracy is acceptable for speech-to-text transcription?
- Should multilingual support be included for AI calling and transcription?
- How will duplicate patient detection be handled across multiple entries?
- What is the expected data retention period for call logs and recordings?
- How will system performance be monitored in production environments?
- What backup frequency should be used for the database?

- How will failures of external services (Twilio, STT APIs) be handled in real-time?
- Should the system support multi-branch or multi-tenant deployment in future?
- What authentication mechanism will be finalized (JWT, OAuth, etc.)?
- How will API rate limiting and abuse prevention be implemented?
- What level of audit logging is required for compliance and tracking?
- How will feature flags be managed and controlled in production?
- What is the rollout strategy for introducing new features to users?
- How will versioning be handled for backward compatibility of APIs?
- What are the SLAs (Service Level Agreements) for system uptime and response time?
- How will sensitive patient data be encrypted and secured at rest and in transit?

15. Architecture Decision Record (ADR)

15.1 Context

MediSched AI must support real-time scheduling, AI-driven outbound calling, speech-to-text processing, and analytics within an 8-week timeline and a small team.

The system needs to:

- Support a relational model for patients, appointments, slots, and doctors
- Provide real-time updates
- Handle asynchronous tasks efficiently
- Maintain security, scalability, and performance

The evaluated options were:

- Node.js + Next.js + MongoDB
- Django REST + React + PostgreSQL
- FastAPI + Next.js + PostgreSQL

15.2 Decision

The chosen architecture is:

Django REST Framework + React.js + PostgreSQL + Celery + Redis

This stack was selected because it provides:

- Strong relational data modeling with PostgreSQL
- Faster development using Django ecosystem
- Reliable background task processing with Celery and Redis
- Support for real-time updates
- Smooth integration of AI capabilities using Python tools

15.3 Consequences

Positive Outcomes

- Faster development and implementation
- Improved data integrity and consistency
- Scalable and modular architecture
- Strong support for AI and analytics features
- Easier long-term maintenance

Tradeoffs / Limitations

- Use of multiple technologies (frontend and backend separation)
- Increased operational complexity due to additional components
- Extra configuration required for real-time features

Future Implications

- Architecture can evolve into microservices if needed
- AI capabilities can be extended further
- Additional integrations can be added without major redesign

16. Approval

16.1 Decision Owner Approval

- **Name:** _____
- **Signature / Approval:** _____
- **Date:** _____

16.2 Design Review Approval

- **Approver(s):** _____
- **Date:** _____
- **Decision:** Approved / Changes Required

17. AI Gold Prompt Mapping (Mandatory)

This section defines the approved AI prompt structure to be used when drafting or refining the PRISM-S Design Document. Free-form prompting is not allowed for PRISM-S artifacts.

17.1 Approved AI Tool(s)

- Primary: ChatGPT
- Secondary (spec-driven refinement): Kiro
- Validation / repo context (optional): Claude Code

17.2 Gold Prompt – PRISM-S

ROLE:
You are a Principal Software Architect at SmartX Technologies.

OBJECTIVE:
Create a complete, implementable design document for the selected solution.

CONTEXT:

- Company: SmartX Technologies

- Project type: Web Application (SaaS Platform) – Healthcare Diagnostic Center
- Selected solution from PRISM-I: Django REST + React + PostgreSQL + Celery + Redis
- Non-functional requirements: Time to Delivery (Critical), Functional Fit (High), NFR Alignment (High), Complexity (High), Cost (Medium), Operational Risk (Medium)
- Coding, security, and observability standards are enforced

INPUTS:

- PRISM-R PRD-Lite (Functional and Non-Functional Requirements)
- PRISM-I Solution Options Matrix
- Architecture constraints and standards (RBAC, encryption, scalability)

INSTRUCTIONS:

- Translate decisions into concrete components, APIs, and workflows
- Explicitly address non-functional requirements, failure modes, and edge cases
- Ensure each requirement maps to a design element and corresponding test
- Avoid vague statements; provide clear, actionable design details

OUTPUT FORMAT:

1. High-level architecture
2. Component-level design
3. Data models and APIs
4. Failure modes and mitigations
5. Test strategy
6. Deployment and rollback plan

QUALITY CHECK:

- Can another developer implement this design without additional clarification?
- Are all requirements traceable to design components?
- Are risks, edge cases, and failure scenarios clearly addressed?

17.3 Evidence for PRISM Gate (M3 → M4)

- Completed PRISM-S Design Document
- ADR included and approved
- Requirements-to-design traceability confirmed
- Design review completed with sign-off